

10.1 Pagamentos: types, helpers e listagem — status de entrega

Bloco 10.1 Pagamentos: types, helpers e listagem · Sprint 10 — Financeiro: Pagamentos · Data: 2026-09-30 · Branch: `main`
· Ambiente: gc-dev (`gzbmhgnpoehormnidmgg`)

Resumo

Implementado, validação pendente para o fechamento da sprint. A listagem de pagamentos está em `/financeiro/pagamentos`, com tudo o que entrou, de qualquer origem, e o total do filtrado no topo. O Financeiro ganhou abas, **Notas fiscais | Pagamentos**, no layout de `/financeiro`: o menu continua com um item só.

O seed `supabase/seed_pagamentos.sql` (atividade do 10.6) foi **adiantado e aplicado em gc-dev**, porque gc-dev só tinha os três pagamentos de NF do seed da sprint 9: a listagem sairia sem acordo e sem avulso.

1. As cinco atividades do bloco

ATIVIDADE DO CLICKUP	ONDE FICOU
Pagamento/Insert/Update e PagamentoListItem com JOIN de obra e NF	<code>src/lib/types.ts</code> . <code>Pagamento</code> passa a tipar <code>origem</code> (nf, acordo, avulso) e <code>forma</code> (com <code>cartao</code> , da migration 004), como <code>NotaFiscal</code> faz com status e tipo. <code>PagamentoListItem</code> traz a obra, a NF (número e série) e a parcela → acordo (número da parcela e descrição), por JOIN
Colunas: Data, Obra, Origem (NF/acordo/avulso), Documento, Forma, Valor	<code>pagamentos-table.tsx</code> , no <code>DataTable</code> . Documento é <code>documentoDoPagamento</code> : "NF 123 / 1", "Sinal outubro · parcela 2" ou "—" no avulso (a observação aparece no título da célula). A NF é link para o detalhe dela; a linha em si não abre nada, porque pagamento não tem tela própria
Busca por obra, número da NF e observação	Obra pelo código ou nome, na lista já carregada; NF pelo número, numa consulta própria que devolve os ids; observação por <code>ilike</code> . Tudo num <code>.or</code> só, porque o PostgREST não mistura coluna da tabela com coluna do JOIN
Filtros: obra, origem, forma, período	O padrão das listagens: estado na URL, busca com debounce de 300 ms, <code>PAGE_SIZE = 20</code> , <code>Pagination</code> e o período por <code>computePeriodoCutoff</code> , sobre a data do pagamento
Totalizador do período filtrado no topo da listagem	Uma segunda consulta, só com o valor, com a mesma função de filtro da lista (<code>filtrar</code>), somada em centavos (<code>totalDosPagamentos</code>). Soma o filtrado inteiro, não a página. Mostra o período e, se houver outro filtro, "com filtros"; também o número de pagamentos

2. Decisões

- **Rota** `/financeiro/pagamentos`, dentro do layout de `/financeiro`, que já tem o guard (admin, financeiro, visualizador). As abas (`financeiro/abas.tsx`) seguem as de `/configuracoes`; a sprint 11 acrescenta a de acordos.
- **Sem botão "Registrar pagamento"** ainda: o formulário é do 10.2, e a baixa pela NF, do 10.3.
- **Sem export:** `/api/export/pagamentos` é do 10.6.
- **JOIN nulo não vira avulso:** se a NF ou a parcela não vier (RLS, por exemplo), a coluna mostra só "NF" ou "Acordo".

3. O seed (adiantado do 10.6)

`supabase/seed_pagamentos.sql`, na obra do SEED-CT-EXEC. Aplicado em gc-dev e conferido pela verificação do próprio arquivo:

PAGAMENTO	ORIGEM	FORMA	VALOR	DOCUMENTO
há 5 dias	avulso	dinheiro	1.500	— (adiantamento em espécie)
há 11 dias	NF	pix	2.000	SEED-NF-003 / 1 (do seed da sprint 9)
há 12 dias	acordo	pix	3.000	SEED-AC-001 Sinal outubro · parcela 1, que o trigger pôs como paga
há 21 dias	NF	boleto	800	SEED-NF-004
há 41 dias	NF	TED	1.200	SEED-NF-004
há 50 dias	avulso	cartão	350	— (taxa de visita técnica)

A parcela 2 do acordo (2.000) fica pendente, sem pagamento. As datas são relativas ao dia em que o seed foi aplicado; por isso o plano não confere totais de período.

4. Plano de validação: estendido, não rodado

CAMADA	ESTADO	O QUE ENTROU
unitário	rodou só <code>pagamentos.test.ts</code> durante o desenvolvimento: 4/4 (a suíte inteira não rodou)	Origens e formas iguais aos CHECKs das migrations (a forma com <code>cartao</code>); guards e rótulo da forma nula; <code>documentoDoPagamento</code> nas três origens e com JOIN nulo; <code>totalDosPagamentos</code> em centavos
estático	<code>tsc --noEmit</code> limpo e <code>next lint</code> sem aviso, durante o desenvolvimento	—

CAMADA	ESTADO	O QUE ENTROU
build	não rodou	+1 rota: <code>/financeiro/pagamentos</code> em <code>rotas-esperadas.txt</code>
runtime	não rodou	13 rotas: a listagem do admin (colunas, as três origens, o totalizador, os filtros, o link da NF); financeiro e visualizador leem; comercial e produção levam 307; <code>origem=acordo</code> (3.000, 1 pagamento); <code>origem=avulso</code> (1.850, 2); <code>forma=cartao</code> (350); busca pelo número da NF (a 004, 2.000 em 2) e pela observação; busca sem resultado com o total zerado; página além do fim; e as abas na listagem de NFs
dados	não rodou	2 checagens, com <code>exigeSeed</code> : a query da listagem (obra, NF e parcela → acordo como objeto, cada origem com o seu documento, as três origens presentes) e a do totalizador (os avulsos somam 1.850 em centavos)
escrita	não rodou	nada novo: a listagem não grava
navegador	não rodou	passo 38, com 2 checagens: a aba Pagamentos a partir das NFs, marcada como atual; o filtro de origem na URL com o totalizador acompanhando. Screenshot <code>38-pagamentos-listagem.png</code>

Durante o desenvolvimento, a query da listagem rodou uma vez contra gc-dev, com a chave de serviço, para conferir que os JOINS de NF e de parcela → acordo resolvem. Isso não é a camada dados, que roda sob RLS com a sessão do admin.

5. Pendências

- O plano não rodou. Fica para o fechamento da sprint (10.6).
- Conferência visual da tela e das abas, no desktop e em 390px, não foi feita.
- O documento de acordo não tem link: a tela do acordo é da sprint 11.

6. Registro para quem for operar prod

Nenhuma migration. `seed_pagamentos.sql` é dado de teste e não vai para prod.